

14. 다음 단계 — CLAUDE.md 너머에 있는 것

이 책에서는 CLAUDE.md의 작성법·발전법·정리법을 다뤘습니다.
마지막 장에서는 CLAUDE.md의 한계와, 그 너머에 있는 Claude Code의 기능을 소개합니다.

CLAUDE.md의 한계 — 대처 수단의 우선순위

1개월 운영에서 깨달은 가장 중요한 것은,
CLAUDE.md의 규칙은 가장 강제력이 낮은 수단이라는 점입니다.
문제에 대처하는 수단에는 강제력의 서열이 있습니다.

우선순위	수단	강제력	예시
1	플로우 내장	최강	코드 안에 검증 로직을 넣는다
2	Hook	강함	파일 저장 후 자동으로 lint 실행
3	함수·스크립트	중간	절차를 스크립트화해서 호출만 하면 되게 한다
4	규칙 (CLAUDE.md)	가장 약함	"~해주세요"라고 쓴다

CLAUDE.md에 "테스트를 반드시 실행할 것"이라고 쓰는 것은 우선순위 4입니다.
컨텍스트가 길어지면 놓치기도 하고, Claude가 "이번에는 불필요하다"라고 판단하는 경우도 있습니다.
반면, Hook으로 "커밋 전에 테스트를 자동 실행"으로 하면 우선순위 2가 되어, Claude의 판단에 관계없이 실행
됩니다.

"~했다면 ~하라"는 규칙이 아니라 **Hook**의 후보. "~를 고려하라"만이 규칙의 영역.
이 구분이 1개월 운영 끝에 도달한 결론입니다.

Hooks: 실제로 동작하는 설정

제 프로젝트에서는 `rules/` 디렉토리의 파일을 편집할 때마다 전 플랫폼에 자동 배포하는 Hook을 사용하고 있습니다.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "bash scripts/auto_sync_rules_hook.sh"
        }]
      }
    ]
  }
}
```

Edit 또는 Write 도구가 사용될 때마다 sync 스크립트가 자동 실행됩니다.

이 스크립트는 `rules/` 하위의 파일이 변경됐는지를 감지하고,

변경이 있으면 전역(`~/.claude/`)과 프로젝트(`.claude/rules/`) 양쪽에 복사합니다.

이것을 CLAUDE.md의 규칙으로 "rules/를 편집하면 sync 스크립트를 실행해주세요"라고 쓸 수도 있습니다.

하지만 그것은 우선순위 4입니다. 실제로 쓰는 것을 잊은 경우가 몇 번 발생해서 Hook으로 이전했습니다.

그 이후, 동기화 누락은 제로입니다.

사용 구분 기준:

- "~를 고려하라" → CLAUDE.md 규칙
- "~했다면 반드시 ~하라" → Hook이 적절
- "~를 금지한다" → Hook으로 감지해서 차단

커스텀 슬래시 커맨드 (Skills)

`.claude/commands/` 디렉토리에 마크다운 파일을 두면,

`/command-name`으로 호출할 수 있는 커스텀 커맨드가 됩니다.

```
<!-- .claude/commands/review.md -->
```

이하의 절차로 코드 리뷰를 수행해주세요:

1. `git diff`로 변경 내용을 확인
2. 보안상의 문제가 없는지 확인
3. 테스트가 추가되어 있는지 확인
4. 개선 제안을 3개 이내로 제시

CLAUDE.md에는 "항상 적용하는 규칙"을, 커스텀 커맨드에는 "필요할 때 호출하는 절차"를 둡니다.

CLAUDE.md에 모든 것을 담으려 하면 비대화되므로, 절차의 정형화에는 커스텀 커맨드가 더 적합합니다.

AGENTS.md: 서브에이전트 전용 지시

AGENTS.md는 Claude Code가 서브에이전트를 시작할 때 읽히는 파일입니다.

CLAUDE.md가 메인 작업자용이라면, AGENTS.md는 서브에이전트용 지시서입니다.

<!-- AGENTS.md -->

- 변경은 최소한으로 한다
- 다른 파일의 변경을 포함하지 않는다
- 완료하면 결과를 간결하게 보고한다

서브에이전트는 독립된 컨텍스트에서 동작하기 때문에, CLAUDE.md 규칙의 일부밖에 보이지 않습니다. 병렬 작업의 규칙(`shared` 디렉토리는 병렬 편집하지 않는다 등)은 AGENTS.md에 쓰는 편이 확실하게 전달됩니다.

CI/CD에서의 CLAUDE.md 활용

CLAUDE.md는 리포지토리에 커밋하는 파일입니다. 즉, CI/CD 파이프라인에서도 활용할 수 있습니다.

활용 예시:

- **PR 리뷰 자동화:** CI에서 Claude Code를 호출해, CLAUDE.md의 규칙에 기반한 코드 리뷰를 실행
- **규칙 정합성 체크:** CLAUDE.md의 행 수나 구조를 CI에서 체크하고 비대화를 감지
- **팀 전체 규약 통일:** CLAUDE.md를 모노레포의 공유 패키지로 관리하고 서브프로젝트에 배포

CLAUDE.md를 코드와 같이 버전 관리함으로써, "언제, 누가, 왜 규칙을 바꿨는가"가 `git log`에 남습니다.

【추가된 내용】한국 CI/CD 환경 보충:

한국 개발 환경에서 자주 사용하는 CI/CD 파이프라인과 연동 방법입니다.

- **GitHub Actions:** `claude` CLI를 Actions 워크플로우에서 호출해 자동 리뷰
- **Jenkins:** Jenkinsfile에서 Claude Code를 실행해 CLAUDE.md 규칙 체크
- **GitLab CI:** `.gitlab-ci.yml`에서 CLAUDE.md의 행 수를 체크해 비대화 경고
- **Jira 연동:** PR 리뷰 결과를 Jira 이슈에 자동 코멘트

다른 도구의 설정 파일과의 비교

CLAUDE.md의 설계 사상은 다른 AI 코딩 도구에도 응용할 수 있습니다.

도구	설정 파일	계층	특징
Claude Code	<code>CLAUDE.md</code> , <code>.claude/rules/</code>	전역 → 프로젝트 → 경로 기반 → Memory	4계층 설계 + Memory 연계
Cursor	<code>.cursor/rules/</code> (구: <code>.cursorsrules</code>)	프로젝트 → 경로 기반	<code>.mdc</code> 형식, glob 패턴으 로 적용 범위 지정
GitHub Copilot	<code>.github/copilot-instructions.md</code>	리포지토리 단위	1파일, Markdown

Cursor는 2025년 후반부터 `.cursor/rules/` 디렉토리에서의 경로 기반 규칙을 지원하고 있으며, Claude Code의 `.claude/rules/` 와 비슷한 설계가 되었습니다.

이 강의에서 배운 테크닉(이유를 쓴다, 조건을 구체화한다, 정기적으로 삭제한다)은 어떤 도구의 설정 파일을 쓸 때도 그대로 사용할 수 있습니다.

이 강의에서 전하고 싶었던 3가지

14개 장을 통해 반복해서 전달해온 메시지는 3가지입니다.

사고에서 쓴다:

"있는 편이 좋을 것 같다"고 쓴 규칙은 사라진다.

"이것이 없으면 망가진다"에서 쓴 규칙만이 살아남는다.

정기적으로 삭제한다:

규칙은 계속 늘어난다. 삭제 기준을 갖지 않으면 파일이 비대화되어 효과가 없어진다.

계속 키워간다:

CLAUDE.md는 완성되는 것이 아니다. 프로젝트가 바뀌면 규칙도 바뀐다.

맺음말

CLAUDE.md는 AI와의 협업의 **"계약서"** 입니다.

무엇을 맡길 것인가. 무엇을 멈출 것인가. 무엇을 기억해둘 것인가.

처음에는 "일단 써보자"로 충분합니다.

제13장의 템플릿을 복사해서 1주일 사용해보세요.

사고가 생기면 규칙을 추가하고, 불필요해지면 삭제한다.

그 사이클을 1개월 돌리면, 여러분의 프로젝트에 최적화된 CLAUDE.md가 성장합니다.

김재우 jaewoo@mdrules.dev